

Université Sorbonne Paris Nord

École d'Ingénieurs Sup Galilée

Base de Données Avancées
TP 5

Ahmed SEHILI

2024/2025

Exercice 1

1. Création de la table

```
CREATE TABLE transaction (  
    idTransaction VARCHAR2(44),  
    valTransaction NUMBER(10)  
);
```

2.

S1:

```
INSERT INTO transaction VALUES ('T1', 5000);
```

S2:

```
SELECT * FROM transaction;
```

La ligne insérée dans S1 n'est pas visible depuis S2 car la trans de S1 n'a pas encore validée.

S1:

```
ROLLBACK;
```

S2:

```
SELECT * FROM transaction;
```

rien de visible. Le ROLLBACK est bon dans S1, il a annulé l'insertion.

S1:

```
INSERT INTO transaction VALUES ('T2', 7500);
```

S2:

```
SELECT * FROM transaction;
```

Résultat observé : La ligne insérée ('T2', 7500) n'est pas là !

Conclusions

- **Session** : C'est juste une connexion entre moi et la base. Chaque session est indépendante.
- **Transaction** : C'est un ensemble d'actions (insert, update...) qu'on veut faire d'un seul coup sans erreur.
- **COMMIT** : Quand je fais COMMIT, tout ce que j'ai fait est enregistré pour de vrai dans la base.
- **ROLLBACK** : Avec ROLLBACK, tout ce que j'ai fait depuis le début de la transaction est annulé, comme si j'avais rien fait.

Exercice 2

1.

```
CREATE TABLE vol (  
    idVol NUMBER(4) PRIMARY KEY,  
    capaciteVol NUMBER(4),  
    nbrPlacesReserveesVol NUMBER(4)  
);  
  
CREATE TABLE client (  
    idClient NUMBER(4) PRIMARY KEY,  
    prenomClient VARCHAR2(50),  
    nbrPlacesReserveesClient NUMBER(4)  
);
```

2.

S1 :

```
INSERT INTO vol VALUES (1, 100, 0);  
INSERT INTO client VALUES (1, 'ahmed', 0);  
COMMIT;
```

S1 : debut de T1

```
UPDATE vol SET nbrPlacesReserveesVol = nbrPlacesReserveesVol + 1 WHERE idVol =  
↪ 1;  
UPDATE client SET nbrPlacesReserveesClient = nbrPlacesReserveesClient + 1  
↪ WHERE idClient = 1;
```

S2 : debut de t2

```
UPDATE vol SET nbrPlacesReserveesVol = nbrPlacesReserveesVol + 1 WHERE idVol =  
↪ 1;  
UPDATE client SET nbrPlacesReserveesClient = nbrPlacesReserveesClient + 1  
↪ WHERE idClient = 1;
```

On voit que S2 est bloqué ? quand il essaye de faire update sur les mm lignes que S1, car S1 n'a pas fait commit.

maintenant S1 fait commit

apres S2 peut continuer

Après le commit de S1, S2 peut terminer son update.

—
Si on ne fait pas attention à l'ordre des transactions, ça peut causer des erreurs ou des blocages. Genre si S1 fait COMMIT mais que S2 pensait qu'il y avait encore l'ancienne valeur, ben c'est pas cohérent.
—

3. SERIALIZABLE

S 1 :

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
UPDATE vol SET nbrPlacesReserveesVol = nbrPlacesReserveesVol + 1 WHERE idVol =
↳ 1;
UPDATE client SET nbrPlacesReserveesClient = nbrPlacesReserveesClient + 1
↳ WHERE idClient = 1;
```

S2 :

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
UPDATE vol SET nbrPlacesReserveesVol = nbrPlacesReserveesVol + 1 WHERE idVol =
↳ 1;
-- ça bloque directement la !
```

Quand on est en serializable, oracle empêche directement que 2 trans modifient les mm données en mm temps. Du coup soit S2 attend que S1 finisse, soit on envoie une erreur.

Quand on met serializable, c'est plus sûr pour éviter les prblms de concurrence, mais c beaucoup plus lent.